

Government AI Cloud

High-Level Vendor-Neutral Reference Architecture and Inception Questionnaire

Prepared for PITB Technical Consultancy Discussion

Prepared by InvoZone

Basis: PITB Corrigendum-01 / Terms of Reference, InvoZone technical proposal, and current technical reference research.



Draft - High-Level Architecture Only

No current PITB infrastructure, GPU quantity, storage platform, network topology, data-center capacity or workload volume is assumed in this document. Final sizing and technology selection require discovery and current-state assessment.

Executive Summary

PITB's requirement should be treated as the design of a sovereign AI computing and operating platform - not merely a GPU cluster and not a single LLM application. The consultancy must define an end-to-end, scalable and vendor-neutral architecture covering compute, storage, orchestration and monitoring; establish measurable performance and scalability benchmarks; and translate the validated design into procurement-ready technical specifications.

Because PITB's current infrastructure is not yet known, the correct starting point is a high-level logical target architecture. It should show how government users, OSSP and shared platforms, APIs, AI services, AI Sandbox environments, model lifecycle controls, CPU/GPU resources, storage, network, security, observability and governance fit together. The architecture should then be refined after the consultant assesses the existing data center, compute estate, network, storage, virtualization/container platforms, security controls, workloads and operational capability.

Core consulting principle: Workload-first, sovereignty-first, multi-model, multi-tenant and modular. PITB should not select the GPU, private-cloud product or LLM first. It should classify workloads and data, establish SLOs, assess reusable infrastructure, benchmark representative workloads and then procure against measurable outcomes.

- Support LLM, Vision AI, Speech AI, Multimodal AI, predictive ML, RAG and future AI workloads through one governed platform.
- Keep sensitive government data on government-controlled infrastructure by default, with controlled hybrid/proprietary model access only where policy permits.
- Separate production AI services from research/sandbox workloads so experimentation cannot compromise production availability or data security.
- Expose models behind internal platform APIs so applications are not permanently tied to one model vendor or one model family.
- Convert the architecture into objective capacity, KPI, acceptance-test and TCO criteria for the later implementation procurement.

What PITB Was Testing in the Last Meeting

PITB Question	Underlying Capability Being Tested
Have we worked on proprietary AI models?	Can the consultant distinguish closed/proprietary services from locally hosted open-weight models, and advise on both without lock-in?
How will we set up the entire infrastructure?	Can the consultant design the full stack from facility, network and GPUs up to orchestration, model serving, MLOps and APIs?
How will we ensure security of data?	Can sovereign government data stay isolated, encrypted, governed and auditable in a shared AI cloud?
How did we use Computer Vision?	Does the team understand non-LLM AI workloads and their different throughput, latency, video and GPU characteristics?
Have we provided the exact solution elsewhere?	Are recommendations grounded in real delivery experience, and can the team distinguish proven component experience from an identical sovereign-cloud deployment?

The response should therefore demonstrate consulting method and architectural judgement, not simply list AI projects or immediately name preferred vendors.

1. High-Level Government AI Cloud Reference Architecture

The following architecture is intentionally logical and vendor-neutral. Technologies such as Kubernetes, OpenShift-class distributions, OpenStack, Ceph, Slurm, vLLM, Triton, KServe, Ray, Prometheus/Grafana and commercial equivalents are candidates for comparative assessment, not pre-selected products.

The high-level design extends the structure already described in the InvoZone proposal - users and government applications above API integration, production AI platform and AI Sandbox above workload services, shared CPU/GPU

resources below them, and security/governance as cross-cutting capabilities - while making the physical data center, network fabric, storage plane and workload scheduling boundaries explicit.

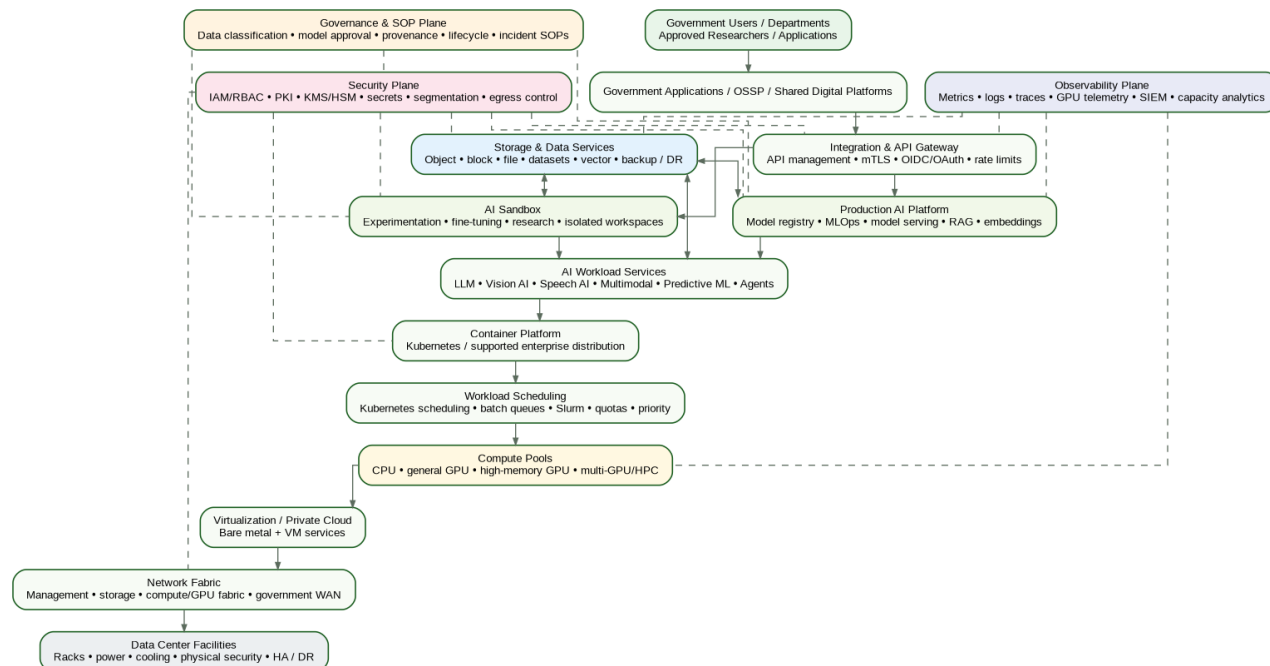


Figure 1. Proposed vendor-neutral logical reference architecture.

1.1 Architecture Layers and Design Intent

Layer	Purpose	Key Consultancy Consideration
Users & Channels	Government departments, approved researchers, citizen-facing applications and machine-to-machine consumers.	Identity, service ownership and data-classification context must travel with each request.
Integration & API Gateway	OSSP/shared-platform integration, API management, authentication, rate limiting, schema validation and service interfaces.	Provides a controlled north-south entry point and keeps downstream AI services decoupled from external applications.
AI Gateway / Model Routing	Internal abstraction over locally hosted and approved external models.	Enables multi-model routing, policy-aware placement, failover, A/B testing and vendor portability.
Production AI Platform	Model registry, MLOps, inference endpoints, RAG, embeddings, reranking, evaluation and controlled deployment.	Hosts production-grade AI services with versioning, approvals, observability and rollback.
AI Sandbox	Isolated experimentation, notebooks, model testing, fine-tuning and research workspaces.	Separates experimentation from production and applies lower-priority quotas, egress controls and promotion gates.
AI Workload Services	LLM, Vision AI, Speech AI, Multimodal, predictive ML and agentic services.	Different workload classes consume different GPU memory, storage and latency profiles.
Container / Orchestration	Kubernetes or a supported enterprise distribution, plus workload policies and device plugins.	Provides portable application scheduling, isolation, service discovery and lifecycle management.
Batch / HPC Scheduling	Kubernetes scheduling and/or Slurm-style queues for long-running and distributed jobs.	Separates latency-sensitive services from batch/fine-tuning/HPC workloads and improves GPU utilization.
Compute Pools	CPU pool, general GPU pool, high-memory GPU pool, and multi-GPU/HPC capacity.	Capability tiers avoid forcing every workload onto the most expensive accelerator class.

Layer	Purpose	Key Consultancy Consideration
Virtualization / Private Cloud	Bare metal where performance requires it; VM/private-cloud services for platform and supporting workloads.	Allows reuse of current investments and isolates management/control-plane services from AI data-plane jobs.
Storage & Data	Object, block and file storage; datasets, model artifacts, vector stores, checkpoints, backups and data-lake capabilities where justified.	AI workloads need both high-throughput sequential access and durable governed repositories.
Network Fabric	Management, storage, compute/GPU fabric, north-south government connectivity and east-west data paths.	Distributed training, model serving and high-throughput storage are often network-bound if fabric is undersized.
Security / Observability / Governance	IAM/RBAC, PKI, KMS/HSM, secrets, segmentation, logging, SIEM, model governance and SOPs.	These controls are cross-cutting; they cannot be bolted on after platform selection.

1.2 Why We Do Not Select Hardware First

PITB's five-year program will contain multiple workload classes with very different resource characteristics. A GPU count is not defensible until the consultant understands model sizes, precision, context lengths, concurrency, training/fine-tuning frequency, CV stream profiles, data volumes, network throughput, availability objectives and growth. Therefore the Week-1 to Week-6 work should progressively turn workload evidence into capacity requirements.

Sizing sequence: Use case inventory -> workload profile -> data classification -> SLO/KPI definition -> existing-capacity assessment -> gap analysis -> benchmark -> target capacity -> phased BoQ.

2. How the Entire Infrastructure Would Be Set Up

At this stage we should present PITB with a deployment blueprint rather than an implementation bill of materials. The implementation sequence below shows how the eventual platform would be built once discovery determines what can be reused and what must be procured.

Stage	High-Level Activity	Why It Matters
1. Facility readiness	Confirm primary/DR sites, rack space, per-rack power, UPS/generator topology, cooling technology, physical security and expansion headroom.	Determines the density and quantity of GPU nodes that can physically be supported.
2. Network foundation	Define redundant management, storage, compute/GPU and government-WAN zones; validate east-west bandwidth and latency.	Avoids a GPU-rich but network-starved platform, especially for distributed AI/HPC and high-throughput storage.
3. Storage/data foundation	Establish governed object/block/file services, model/dataset repositories, vector/data-lake capability as required, backup and DR.	Separates durable data/model assets from ephemeral compute and provides recovery and lineage.
4. Private cloud / bare metal	Reuse or establish VM/private-cloud services and bare-metal AI nodes according to performance needs.	Balances AI performance with operational manageability and existing investments.
5. Container & scheduling plane	Deploy Kubernetes/enterprise distribution, accelerator device plugins, quotas, priorities and - where justified - an HPC scheduler.	Creates controlled resource sharing across departments and workload classes.
6. AI platform plane	Deploy model registry, approved artifact repository, MLOps pipelines, serving engines, RAG/embedding services and model gateway.	Makes model lifecycle and deployment repeatable instead of project-specific.
7. Sandbox & promotion path	Create isolated research workspaces with egress controls, lower-priority GPU quotas,	Enables innovation without exposing production credentials or restricted

Stage	High-Level Activity	Why It Matters
	expiries and production promotion gates.	government data.
8. Integration plane	Connect OSSP/shared digital platforms through API gateway, government identity federation and approved service interfaces.	Prevents point-to-point model integrations and centralizes policy enforcement.
9. Security & observability	Integrate IAM/PAM/PKI/KMS/HSM/secrets, segmentation, container/model supply-chain controls, logs, metrics, SIEM and incident response.	Provides evidence that sovereignty and tenant isolation are enforceable.
10. Acceptance & handover	Run capacity, failover, isolation, security, RAG authorization, model-serving, storage/network and DR tests.	Turns vendor promises into measurable procurement acceptance criteria.

3. Data Security, Sovereignty and Multi-Tenancy

Security should begin with data classification and workload placement rather than with a firewall product. The SOW explicitly identifies sovereignty, privacy, regulatory and compliance concerns around sensitive government data. The default design therefore keeps sensitive workloads on government-controlled infrastructure and treats any external proprietary model endpoint as an optional, policy-controlled integration.

3.1 Proposed Trust and Security Zones

- External / government application ingress zone: API gateway, WAF/rate limiting and authenticated application interfaces.
- Management zone: infrastructure management, hypervisors/private cloud, Kubernetes control plane, BMC/OOB management and admin jump access.
- Production AI zone: model-serving endpoints, RAG services, embeddings and production application services.
- Sandbox / research zone: isolated experimentation with stricter egress and credential restrictions.
- Compute/GPU zone: tenant-aware GPU/CPU worker pools with quotas, node pools and workload policies.
- Storage/data zone: governed object/block/file/vector repositories with tenant/data-class ACLs and encryption.
- Security/observability zone: SIEM, audit, metrics, vulnerability management, secrets/KMS/HSM and incident tooling.
- Backup/DR zone: independent recovery copies and replicated services according to approved RPO/RTO.

3.2 Control Model

Security Domain	High-Level Control
Identity & access	Federated government identity, MFA, RBAC/ABAC where needed, service identities, privileged access management and periodic access review.
Network isolation	Segment management, storage, compute, production, sandbox and ingress; default-deny east-west policy where feasible; controlled egress.
Encryption & keys	Encryption in transit and at rest; government-controlled KMS/HSM; key rotation/revocation; separate key administration from AI platform administration.
Tenant isolation	Namespace/project quotas for ordinary tenants, separate node pools for stronger isolation, and dedicated nodes/clusters for highly restricted workloads.
RAG authorization	Every source retains owner, classification and ACL metadata; authorization is enforced before retrieved context reaches the model.
Model supply chain	Approved internal model/artifact registry, hashes/signatures, license record, vulnerability review, provenance and controlled production promotion.
Fine-tuning security	Dataset lineage, base-model approval, isolated training workspaces,

Security Domain	High-Level Control
	checkpoint protection, no uncontrolled telemetry/Internet for sensitive jobs.
Audit & SIEM	Log authentication, authorization, admin changes, model deployment, dataset access and security events; prompt/response logging must be classification-aware.
Incident response	Playbooks for cross-tenant exposure, compromised model artifacts, credential/key compromise, malicious models, prompt/data leakage and service abuse.

Multi-tenancy principle: Sharing the same physical AI Cloud does not mean sharing data, models, credentials, vector indexes, logs or administrative authority. Resource sharing and data isolation are designed separately.

4. Representative End-to-End Workload Flows

4.1 Government RAG / LLM Query

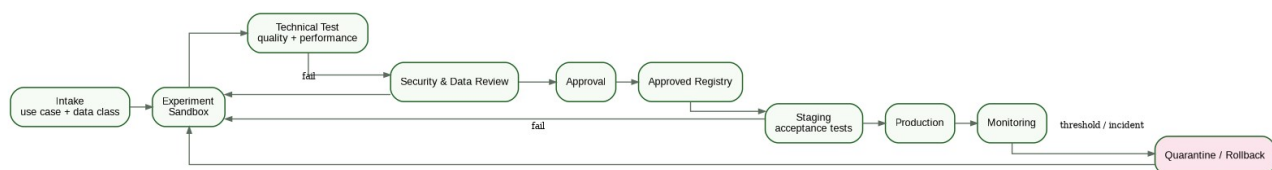
1. User or application authenticates through the government identity boundary and calls the API gateway.
2. Policy engine determines tenant, data classification and approved model class for the request.
3. RAG retrieval searches only data the user/application is authorized to access; source ACLs are applied before context is assembled.
4. AI Gateway routes the request to an approved locally hosted model, or to a controlled external model only if policy permits that data class.
5. Model response passes output/safety checks and is returned with source references where the application requires grounding.
6. Telemetry records request metadata, model/version, retrieval sources, latency, policy decision and security events without unnecessarily duplicating sensitive prompt content.

4.2 Computer Vision Inference

7. Approved camera/image/video source enters through a controlled ingestion interface; stream and source identity are authenticated.
8. CPU/video preprocessing performs decode, resizing, frame sampling and normalization; this prevents wasting expensive GPU cycles on non-accelerated tasks.
9. Vision workload is scheduled to an appropriate GPU pool based on FPS, latency, resolution and model requirements.
10. Inference produces detections/classifications/embeddings; business rules convert these into events for downstream government applications.
11. Only required frames/events are retained according to data-retention policy; full video storage is not assumed unless the use case requires it.
12. GPU utilization, FPS, p95 inference latency, dropped-frame rate and source-to-alert latency become acceptance-test metrics.

This is how InvoZone should connect its documented Computer Vision experience (for example CCTV-based real-time detection/anomaly patterns and liveness/face-comparison workloads) to the Government AI Cloud architecture: CV is a first-class workload with different ingest, latency and throughput behavior from LLM inference.

4.3 Experimentation to Production



Nothing should move directly from a research notebook into production. Every production model should have a registry record, evaluation evidence, security/data review, approved serving configuration, staged acceptance test and rollback path.

5. Multi-LLM Strategy and Differentiation

PITB should not select one permanent 'Government LLM'. The platform should expose an internal model abstraction layer so models can be benchmarked and replaced as quality, licensing, hardware economics and policy evolve. Sensitive workloads should default to locally hosted models; proprietary managed services should remain optional and classification-aware.

Model / Type	Deployment Position	Why It May Fit	Key Caveat for PITB
Llama family	Open-weight/local deployment	Broad ecosystem; good candidate for sovereign self-hosting; multiple sizes and quantization options.	License and hardware needs vary by release; Urdu quality must be benchmarked, not assumed.
Qwen family	Open-weight/local deployment	Wide model-size range; strong multilingual positioning; attractive for local multilingual evaluation.	Model architecture and memory needs vary; formal Urdu/Roman Urdu/code-switching require PITB-specific tests.
Mistral family	Mixed open-weight/commercial	Efficient model options, local deployment paths, general and specialist models.	Licensing differs by exact model; do not generalize one license across the family.
Falcon family	Open/open-access candidates	Useful additional sovereign option; multilingual and efficiency-focused variants exist.	Ecosystem maturity and exact model quality must be measured against stronger alternatives; license is release-specific.
GPT family	Proprietary managed service	Strong general/reasoning/multimodal benchmark; rapid capability evolution.	External dependency, recurring usage cost, residency/retention and sovereign-data policy must be explicitly approved.
Claude family	Proprietary managed service	Strong document/reasoning and large-context use cases.	Managed service boundary and regional/data-processing terms require review for government data.
Gemini family	Proprietary managed service	Strong multimodal/voice-oriented capabilities and broad model portfolio.	External service boundary and production-version lifecycle must be governed.
Smaller specialist models	Local or managed	Embeddings, reranking, OCR, classification, extraction and safety tasks can be cheaper and faster than a large general LLM.	Requires orchestration and evaluation discipline so applications use the right model for the task.

5.1 PITB-Specific Model Benchmark

Benchmark Domain	Representative Test	Primary Metrics
Urdu / English instruction following	Formal Urdu, English, Roman Urdu and code-switching	Human task score; adherence; error rate
Government-domain QA	Policies, notifications, procedures and service information	Correctness; abstention; unsupported-claim rate
RAG grounding	Answers constrained to authorized government documents	Grounded-answer rate; source precision; retrieval recall; hallucination rate
Document extraction	Forms, letters, tables and structured records	Field F1/exact match; schema-valid output rate
Summarization / translation	Long Urdu/English government documents	Factual consistency; coverage; bilingual

Benchmark Domain	Representative Test	Primary Metrics
	and terminology	terminology consistency
Safety / privacy	Prompt injection, restricted data and PII challenges	Attack success rate; prohibited-disclosure rate; correct refusal rate
Long context	Large policy and case-document sets	Answer accuracy vs context size; latency; memory impact
Performance	Controlled prompt/output lengths and concurrency	TTFT p50/p95; inter-token latency; tokens/s; requests/s; GPU memory
Operational fit	Deploy, upgrade, rollback, quantize and fine-tune	Deployment time; compatibility; rollback/recovery; operator complexity
Economics	Same benchmark set across candidates	Five-year local TCO or managed cost per standardized workload

Urdu requirement: A model being described as multilingual is not enough. PITB should own a versioned Urdu/English evaluation corpus that includes formal Urdu, Roman Urdu, government terminology, RAG cases and difficult code-switching examples.

6. GPU / Capacity Sizing Methodology

The architecture should define capability tiers rather than make up GPU quantities. Actual sizing emerges from representative workload profiles and measured benchmarks.

Workload Class	Typical Behavior	Architecture Implication
Interactive LLM inference	Latency-sensitive, variable context, continuous traffic	High VRAM, efficient batching, reserved production capacity
RAG / embeddings / reranking	High document/query volume, often smaller models	Smaller accelerators may suffice; storage/vector latency matters
Vision / video inference	Continuous frames/streams, FPS and source-to-alert SLOs	Throughput-oriented GPUs plus strong ingest/network capacity
Speech	Streaming or batch audio	Low-latency inference and appropriate audio preprocessing
Multimodal	Larger memory footprint plus preprocessing	High-memory accelerators plus CPU and storage bandwidth
Fine-tuning	Bursty, long-running, checkpoint-heavy	Multi-GPU pool, high storage throughput, batch scheduling
Training / HPC	Distributed communication intensive	High-bandwidth scale-up/scale-out fabric and HPC scheduler become important
Sandbox / research	Unpredictable, many short-lived jobs	Quotas, preemption, lower priority and strong tenant isolation

A useful planning sanity check is raw parameter memory approximately equals parameter count multiplied by bits per parameter divided by eight. This is only the weight-storage floor. Production capacity must also account for KV cache, runtime tensors, batch/concurrency, context length, serving overhead, HA reserve and fragmentation.

- General inference tier: smaller models, embeddings, reranking, CV and experimentation.
- High-memory inference tier: larger LLMs, longer contexts and higher concurrency.
- Scale-up / multi-GPU tier: larger models, fine-tuning and HPC/distributed workloads.
- Reserved production capacity: protected from opportunistic sandbox jobs.
- Headroom and HA reserve: capacity planned for maintenance/failure rather than 100% nominal utilization.

6.1 Capacity Inputs We Need from PITB

- Model families/sizes expected in the first phase, expected prompt/context ranges and production concurrency.
- Target p95/p99 response latency or TTFT, throughput and service availability by workload class.
- Number, resolution and FPS of Computer Vision streams, plus acceptable dropped-frame and source-to-alert latency.
- Expected speech concurrency, audio duration and real-time vs batch requirements.
- Fine-tuning/training frequency, dataset sizes, checkpoint behavior and maximum acceptable queue times.
- Storage volume, dataset growth, read/write throughput, backup/retention and DR requirements.
- One-, three- and five-year growth assumptions and required capacity headroom.

7. KPI and Acceptance-Test Framework

Exact thresholds should not be invented before discovery. However, the consultancy can define the KPI families now so PITB knows how the eventual implementation will be validated.

KPI Family	Example Acceptance Measures
AI serving	TTFT, p50/p95/p99 latency, tokens/s, requests/s, model load time, failed-request rate
Computer Vision	FPS, p95 inference latency, dropped-frame rate, source-to-alert latency, GPU utilization
GPU / scheduling	GPU and VRAM utilization, queue time, quota enforcement, scheduling latency, recovery after node failure
Storage	Read/write throughput, IOPS/latency by test profile, rebuild time, backup success and restore validation
Network	Effective throughput, p95 latency, packet loss, fabric failover, storage-to-GPU transfer rate
Platform	Control-plane availability, API p95 latency, workload recovery, deployment/rollback time
Security	MFA/RBAC tests, cross-tenant isolation, key rotation/revocation, audit completeness, vulnerability remediation
DR / continuity	RPO/RTO by service class, restore success, failover and recovery drills
Operations	Alert detection time, incident acknowledgement/restoration, capacity threshold reporting

8. Procurement-Ready Outputs

The reference architecture must ultimately become a PPRA-ready implementation package. The key principle is to specify functional outcomes and measurable performance rather than writing a preferred vendor's product sheet into the RFP.

BoQ Category	What the Specification Should Capture
Data-center augmentation	Rack, power, cooling, PDUs, cabling and facility work where existing capacity is insufficient
Compute management nodes	CPU, memory, boot/storage, redundant network and management requirements
GPU/accelerator nodes	Minimum memory/performance class, interconnect, secure partition/isolation capability, support and lifecycle
HPC/high-memory nodes	Scale-up topology, accelerator-to-accelerator/inter-node bandwidth, CPU/RAM balance
Network fabric	Compute, storage and management fabrics, optics/cables,

BoQ Category	What the Specification Should Capture
	redundancy and oversubscription limits
Storage	Usable object/block/file capacity, performance, data protection, replication/erasure, growth
Backup / DR	Independent copies, retention, restore tests, DR replication and recovery objectives
Private cloud / orchestration	Virtualization/private cloud, Kubernetes distribution/support, registry, networking and policy components
AI platform	Model registry, MLOps, serving, RAG/embedding interfaces and approved internal repositories
Security / observability	IAM integration, PAM/KMS/HSM/secrets, PKI, SIEM, metrics/logging/tracing and GPU telemetry
Services / knowledge transfer	Installation, configuration, hardening, test execution, documentation, admin/operator/developer training
Support	Warranty, spares, support response, software lifecycle, upgrade and maintenance commitments

TCO should cover the full program horizon, including accelerator/server/storage/network CapEx, software/support, power, cooling, facility changes, spares, warranty extension, operations staff, training, refresh cycles, DR, security tooling, model licenses and - where permitted - proprietary API/service costs.

9. Direct Answers to PITB's Five Meeting Questions

9.1 Have We Worked on Proprietary AI Models?

The answer must distinguish two meanings. Working with proprietary model services is different from training a proprietary foundation model. The consultancy position should be that InvoZone understands how proprietary model APIs and locally hosted/open-weight models are integrated, benchmarked and governed, but we should not claim that InvoZone trained a proprietary foundation model from scratch unless documentary project evidence exists.

For PITB, proprietary models should be treated as optional benchmark/integration candidates. The architecture remains local-first for sensitive data and avoids hard-coding applications to any external provider.

9.2 How Would We Set Up the Entire Infrastructure?

We would follow the staged blueprint in Section 2: validate facilities -> network -> storage/data -> private cloud/bare metal -> Kubernetes/HPC scheduling -> AI platform -> Sandbox -> OSS/API integration -> security/observability -> acceptance testing. Exact products and quantities are selected only after current-state assessment and workload benchmarking.

9.3 How Will We Ensure Security of Data?

Through classification-driven workload placement, sovereign hosting for sensitive data, government identity federation, least-privilege RBAC, strong tenant isolation, network segmentation, encryption, government-controlled keys, secrets management, RAG ACL enforcement, approved model artifacts, controlled fine-tuning, full audit integration and incident response. Security is a cross-cutting architecture property rather than one appliance.

9.4 How Did We Use Computer Vision?

The InvoZone proposal documents Computer Vision workloads such as CCTV-based real-time detection/anomaly patterns and face-comparison/liveness checks. For PITB, these examples demonstrate understanding of continuous video/image ingestion, preprocessing, GPU inference, low-latency event generation and different capacity metrics from LLM workloads. The platform therefore requires workload-aware GPU pools rather than one generic AI server.

9.5 Have We Provided the Exact Solution Elsewhere?

We should not claim an identical sovereign Government AI Cloud unless documentary evidence proves it. The credible answer is that InvoZone has experience across relevant building blocks - AI/LLM applications, Computer Vision, cloud, containers, data and production AI - and will apply proven architecture patterns while designing around PITB's actual infrastructure, sovereignty constraints and workload profile. Any directly comparable AI/HPC/private-cloud architecture project or JV-partner evidence should be presented separately with documentary proof.

10. Five High-Value Technical Questions for PITB

These questions are intentionally deeper than 'how many servers do you have?'. Each one materially changes the target architecture and demonstrates that the consultancy understands the relationship between facilities, workloads, sovereignty, platform operations and performance.

1. What physical data-center and high-speed fabric constraints must the AI Cloud fit within?

Evidence we need: Please share primary/DR site details, free rack space, allowable kW per rack, UPS/generator redundancy, cooling type/capacity, current switch speeds/topology, storage/compute fabrics, oversubscription and any RDMA/InfiniBand or equivalent capability.

Architecture decision it drives: GPU density and distributed AI performance can be limited by power, cooling and east-west bandwidth before compute itself becomes the bottleneck. This decides whether we can reuse the current facility/fabric or need augmentation.

2. What is the first-phase workload envelope and its service-level objectives?

Evidence we need: For each LLM/RAG, Vision, Speech, Multimodal and fine-tuning use case, provide model-size expectations, context ranges, concurrency/TPS, p95/p99 latency or TTFT, CV stream count/FPS/resolution, speech concurrency, batch/training windows, Sandbox users and 1/3/5-year growth.

Architecture decision it drives: This is the primary input to GPU memory, pool composition, scheduling, reserved capacity, storage throughput and five-year scalability.

3. What are the authoritative sovereignty, data-classification and tenant-isolation rules?

Evidence we need: Which data classes may never leave government infrastructure? Which may use externally managed proprietary AI? Do any departments require dedicated nodes/clusters? What rules apply to academia/startup Sandbox users, and what prompt/response logging is allowed?

Architecture decision it drives: This determines local-vs-external model placement, cluster topology, egress control, RAG index separation, encryption/key boundaries and whether namespace, node or cluster isolation is required.

4. Which identity, security and operations platforms must the AI Cloud integrate with rather than duplicate?

Evidence we need: Please provide current AD/LDAP/SSO/MFA/PAM, PKI, KMS/HSM, SIEM/SOC, firewall/WAF, secrets, monitoring/logging, vulnerability management, incident-response ownership, log-retention requirements and disconnected-operation constraints.

Architecture decision it drives: This defines the control plane, zero-trust integration, audit model, privileged operations and the amount of new tooling that should or should not be procured.

5. What compute, storage, virtualization/container and data platforms are reusable today, and what performance do they actually deliver?

Evidence we need: Please share server/GPU inventory, VRAM, age/warranty/utilization, VMware/OpenStack/Nutanix or other private-cloud stack, Kubernetes/OpenShift version/topology, container registry, Slurm/HPC scheduling if any, SAN/NAS/object platforms, usable capacity, IOPS/throughput, backup/DR and OSSP/data-platform interfaces.

Architecture decision it drives: This directly drives current-state reuse, gap analysis, private-cloud design, Kubernetes/HPC integration, storage architecture, TCO and the eventual BoQ.

Additional policy question: Which government data classifications, if any, may be sent to an externally managed proprietary AI service, and under what residency, retention, encryption, contractual and logging conditions? Until PITB answers this, external proprietary models should be optional controlled integrations, not architectural dependencies.

11. Current Unknowns and Working Assumptions

Unknown / Risk	Inception Assumption	Why It Matters
Existing compute/GPU capacity unknown	No GPU quantity or model is assumed	No defensible BoQ until inventory and benchmark evidence exist
Power/rack/cooling headroom unknown	Existing DC may require augmentation	Could make proposed accelerator density physically impossible
Target workload volumes unknown	Architecture supports workload classes conceptually	Cannot finalize throughput, SLA or capacity
Security/data-class policy not supplied	Sensitive workloads default to sovereign local hosting	Cannot finalize external model/hybrid policy
OSSP/API details unknown	Open API integration layer assumed	Identity, throughput and network design remain provisional
HA/DR targets undefined	Critical production services assumed redundant	Large CapEx/TCO variance depending on RPO/RTO
Tenant trust boundaries unknown	Departments treated as separate logical tenants	Cannot finalize namespace vs node vs cluster isolation
Operations capability unknown	Supportability is a major selection criterion	Overly complex open-source stack could be operationally unsustainable
Model licensing/version turnover	Exact model/version evaluated at approval freeze	Legal and operational risk if a whole family is treated as one static product
Hardware availability/lead times unknown	Equivalent capability classes should be allowed	Reduces supply-chain and procurement-delay risk

12. Recommended Immediate Consulting Actions

1. Issue the five-question evidence request before the infrastructure workshop and ask PITB for diagrams/inventories rather than verbal answers only.
2. Create 5-8 representative workload profiles: Urdu/English government RAG, high-concurrency LLM inference, document intelligence, real-time Computer Vision, Speech/voice, Multimodal, one fine-tuning job and one Sandbox/research profile.
3. Agree the sovereignty decision matrix: which data classes are local-only, which may use controlled proprietary services, and who approves exceptions.
4. Create the benchmark corpus and acceptance methodology before selecting GPUs or LLMs. Public benchmarks shortlist candidates; PITB workloads determine suitability.
5. Produce Architecture Baseline v0.1 plus an Architecture Decision Register covering platform, scheduler, storage, GPU tiers, tenant isolation, external-model policy, observability and DR.
6. Assemble evidence-backed InvoZone experience: separate use of proprietary model services from proprietary model training; use verified Computer Vision examples; provide comparable cloud/HPC evidence only where documentary proof exists.

Recommended meeting message: InvoZone will not prescribe a GPU, LLM or cloud product before understanding Punjab's workloads and infrastructure. We will establish the current-state baseline, classify data and sovereignty requirements, benchmark representative AI workloads, design a modular multi-tenant architecture, define measurable acceptance thresholds and then translate the validated design into a vendor-neutral BoQ and PPRA-ready RFP.

Reference Sources

Primary project source: PITB Corrigendum-01 / Terms of Reference - Technical Consultancy for Establishing a Government AI Cloud to Enable Advanced Digital Technologies through EDM & AI-Driven Digital Services.

Proposal source: InvoZone Technical Proposal - RFP#106082026-1, including the proposed high-level Government AI Cloud architecture, technology comparison categories, governance and delivery methodology.

- NIST AI Risk Management Framework and Generative AI Profile - AI governance and risk-management reference.
- NIST SP 800-207 - Zero Trust Architecture.
- NIST SP 800-53 Rev. 5 - security and privacy control families.
- NIST SP 800-61 Rev. 3 - incident response guidance.
- Kubernetes documentation - GPU scheduling, multi-tenancy and resource quotas.
- Slurm documentation - accelerator/GPU scheduling through Generic Resources (GRES).
- Ceph documentation - distributed object, block and file storage patterns.
- OpenTelemetry, Prometheus and Grafana documentation - portable telemetry, metrics and dashboards.
- vLLM documentation - open LLM serving architecture and throughput-oriented serving patterns.
- Official model documentation/repositories for Meta Llama, Qwen, Mistral and Falcon, plus current proprietary-model documentation for OpenAI, Anthropic and Google Gemini.

Note: All named technologies and model families in this document are assessment candidates or reference examples. Final selection must be evidence-based, workload-specific, vendor-neutral and approved through PITB's consultancy and procurement process.